

Text and Image Are Mutually Beneficial (TIMO) per il Few-Shot Classification Training-Free con CLIP e dataset usato

Basato sul paper di Yayuan Li, Jintao Guo, Lei Qi, Wenbin Li, Yinghuan Shi

16 maggio 2025

Sommario

Questo documento descrive TIMO (Text-Image Mutual guidance Optimization) e la sua variante migliorata TIMO-S, metodi proposti per migliorare il Few-Shot Learning (FSL) training-free utilizzando il modello CLIP (Contrastive Language-Image Pretraining). L'obiettivo è affrontare due problemi chiave derivanti dalla modellazione indipendente delle modalità testuale e visiva: (1) un severo disallineamento anomalo nella modalità immagine e (2) una qualità variabile dei prompt testuali generati. TIMO introduce un meccanismo di guida reciproca tra testo e immagine per mitigare questi problemi.

Indice

1	Introduzione: Problemi nel Few-Shot Learning con CLIP	3
2	Preliminari: CLIP e Classificazione Few-Shot	3
3	TIMO: Ottimizzazione a Guida Mutua Testo-Immagine	3
3.1	Text-Guided Image (TGI): Guida Testuale per l'Immagine	4
3.2	Image-Guided Text (IGT): Guida Visuale per il Testo	4
3.3	Combinazione in TIMO	5
4	TIMO-S: Versione Migliorata di TIMO	5
5	Dataset Utilizzato	5
5.1	Composizione del Dataset Originale	5
5.2	Creazione del Nuovo Dataset per la Sperimentazione	5
5.3	Ruolo dei Prompt Testuali Generati da LLM	6
6	Risultati	9
6.1	Tabella	9
6.2	Analisi dei Risultati	10
A	Pseudo-codice di TIMO	12

1 Introduzione: Problemi nel Few-Shot Learning con CLIP

CLIP ha dimostrato notevoli capacità di generalizzazione zero-shot e si è affermato come una base potente per il Few-Shot Learning (FSL). Tuttavia, molti metodi FSL basati su CLIP che non richiedono addestramento aggiuntivo (training-free) tendono a trattare le modalità testuale e visiva in modo indipendente. Questo approccio presenta due limiti principali:

1. **Disallineamento Anomalo nella Modalità Immagine (Anomalous Match):** Durante il pre-addestramento di CLIP, possono emergere delle "scorciatoie" o bias che portano a una somiglianza elevata tra rappresentazioni di immagini appartenenti a classi diverse. Questo problema è particolarmente pronunciato in FSL, dove il numero limitato di campioni di supporto può portare il modello a fare previsioni errate con alta confidenza. La modalità visiva, essendo più ricca di informazioni rispetto a quella testuale, è più suscettibile a queste imperfezioni intrinseche di CLIP.
2. **Qualità Variabile dei Prompt Testuali Generati:** I metodi training-free spesso si affidano a prompt testuali generati automaticamente (ad esempio, da Large Language Models) per descrivere le classi. Tuttavia, la qualità di questi prompt può variare significativamente. Alcuni prompt possono essere poco descrittivi, semanticamente distanti dalle caratteristiche visive delle immagini, o addirittura fuorvianti. Questo divario semantico tra le rappresentazioni testuali e visive può portare a classificatori sub-ottimali, specialmente quando si considerano più prompt per classe. Il paper evidenzia come i prompt peggiori possano degradare significativamente le prestazioni.

Per affrontare congiuntamente questi problemi, TIMO propone un meccanismo di **guida reciproca (mutual guidance)** tra testo e immagine.

2 Preliminari: CLIP e Classificazione Few-Shot

CLIP impara uno spazio di embedding congiunto in cui le rappresentazioni di immagini e testi semanticamente simili sono vicine.

- Data una query immagine, CLIP ne estrae la feature visiva $\mathbf{f}_v \in \mathbb{R}^D$.
- Per ogni classe i (su N classi), si generano P prompt testuali (es. "una foto di una [CLASSE]"). Questi vengono codificati da CLIP per ottenere le feature testuali $\mathbf{F}_t \in \mathbb{R}^{N \times P \times D}$. Solitamente, si calcola una media dei P prompt per ottenere un prototipo testuale per classe $\mathbf{W}_t \in \mathbb{R}^{N \times D}$.
- In FSL, si dispone di K immagini di supporto per classe. Queste vengono codificate per ottenere le feature visive di supporto $\mathbf{F}_v \in \mathbb{R}^{N \times K \times D}$, da cui si possono derivare prototipi visivi per classe $\mathbf{W}_v \in \mathbb{R}^{N \times D}$.

La classificazione zero-shot standard in CLIP calcola i logits come la similarità coseno tra $\mathbf{f}_v$ e $\mathbf{W}_t$. I metodi FSL training-free spesso combinano i logits testuali (zero-shot) con i logits derivati da un classificatore basato sulle immagini di supporto (es. $\text{Classifier}_{\text{image}}(\mathbf{f}_v)$), tipicamente tramite una somma pesata:

$$\text{logits} = \text{logits}_{\text{text}} + \alpha \cdot \text{logits}_{\text{image}}$$

Tuttavia, come discusso, la costruzione indipendente di questi classificatori è sub-ottimale.

3 TIMO: Ottimizzazione a Guida Mutua Testo-Immagine

TIMO introduce due componenti chiave: Image-Guided Text (IGT) e Text-Guided Image (TGI), che lavorano sinergicamente.

3.1 Text-Guided Image (TGI): Guida Testuale per l'Immagine

Obiettivo: Mitigare il problema del disallineamento anomalo nella modalità immagine, sfruttando le rappresentazioni testuali, generalmente più robuste. **Meccanismo:**

1. Per ogni classe i e per ciascuno dei suoi P prompt testuali (feature $F_t^{i,p}$), si calcola una matrice di pesi $s^{i,p}$ basata sulla similarità coseno tra il prompt $F_t^{i,p}$ e il prototipo visivo della classe W_v^i :

$$s^{i,p} = \frac{F_t^{i,p} \cdot (W_v^i)^T}{\|F_t^{i,p}\| \|W_v^i\|}$$

Questo peso indica quanto un prompt testuale è rilevante per le caratteristiche visive della classe.

2. Si introduce un iperparametro β che controlla il numero di prompt testuali da considerare per la guida. I pesi $s^{i,p}$ vengono utilizzati per selezionare/pesare i prompt più rilevanti. Sia $S \in \mathbb{R}^{N \times P}$ la matrice di questi pesi (eventualmente con zero per i prompt scartati).
3. Le feature di supporto visive $\mathbf{F}_v$ vengono combinate con le feature testuali pesate $F_t \odot S$ (prodotto element-wise, o una selezione basata su β) per ottenere una rappresentazione arricchita $\mathbf{F}_{\text{TGI}}$:

$$\mathbf{F}_{\text{TGI}} = \text{Concat}(\mathbf{F}_v, \mathbf{F}_t \odot S) \in \mathbb{R}^{N \times (K + \beta') \times D}$$

dove β' è il numero effettivo di prompt testuali usati dopo la selezione.

4. Un classificatore per immagini, $\text{Classifier}_{\text{TGI}}$, viene costruito utilizzando queste feature $\mathbf{F}_{\text{TGI}}$ (ad esempio, seguendo la logica di Tip-Adapter o GDA-CLIP, che il paper chiama 'ImageClassifierBuilder').
5. I logits TGI per l'immagine di query $\mathbf{f}_v$ sono: $\text{logits}_{\text{TGI}} = \text{Classifier}_{\text{TGI}}(\mathbf{f}_v)$.

In sostanza, TGI "raffina" le feature visive di supporto includendo le informazioni testuali più allineate, con l'obiettivo di creare un classificatore di immagini più robusto ai disallineamenti.

3.2 Image-Guided Text (IGT): Guida Visuale per il Testo

Obiettivo: Rettificare la qualità variabile dei prompt testuali, utilizzando le rappresentazioni delle immagini come guida per selezionare o pesare i prompt in modo più efficace. **Meccanismo:** Dal punto di vista dell'ensemble learning, un buon classificatore IGT deve soddisfare due obiettivi:

1. **Target I:** Classificare correttamente i dati.
2. **Target II:** Mantenere l'individualità (diversità) delle informazioni originali dei prompt.

Per fare ciò:

1. Per ogni classe i , si cerca un vettore di pesi $r^i \in \mathbb{R}^P$ per i P prompt testuali F_t^i che massimizzi l'allineamento con il prototipo visivo W_v^i della classe (dopo normalizzazione L2 di tutte le feature):

$$\max_{r^i} (r^i)^T F_t^i (W_v^i)^T$$

soggetto al vincolo che la norma di r^i sia controllata da un iperparametro γ : $\|r^i\| = \gamma$. Questo vincolo aiuta a preservare la diversità dei prompt (Target II).

2. La soluzione ottimale per r^i è proporzionale a $F_t^i (W_v^i)^T$. Questi pesi r^i vengono poi normalizzati (ad es. con Softmax) per ottenere una distribuzione R^i sui prompt. Sia $R \in \mathbb{R}^{N \times P}$ la matrice di questi pesi combinati per tutte le classi.

3. Le feature testuali originali $\mathbf{F}_t$ vengono trasformate utilizzando questi pesi per ottenere le feature testuali rettificate $\mathbf{F}_{\text{IGT}}$:

$$\mathbf{F}_{\text{IGT}} = \mathbf{F}_t R^T \in \mathbb{R}^{N \times D} \quad (\text{o } \mathbf{F}_{\text{IGT}} = \text{einsum}(\text{"np, npd} \rightarrow \text{nd"}, R, \mathbf{F}_t))$$

4. Un classificatore testuale, $\text{Classifier}_{\text{IGT}}$, viene costruito utilizzando queste feature $\mathbf{F}_{\text{IGT}}$ (che il paper chiama ‘TextClassifierBuilder’).
5. I logits IGT per l’immagine di query $\mathbf{f}_v$ sono: $\text{logits}_{\text{IGT}} = \text{Classifier}_{\text{IGT}}(\mathbf{f}_v)$.

IGT quindi ri-pesa o combina linearmente i prompt testuali per ogni classe in modo che siano più semanticamente allineati con le caratteristiche visive di quella classe.

3.3 Combinazione in TIMO

TIMO combina i logits ottenuti dai classificatori TGI e IGT tramite una somma pesata, controllata da un iperparametro α :

$$\text{logits}_{\text{TIMO}} = \text{logits}_{\text{TGI}} + \alpha \cdot \text{logits}_{\text{IGT}}$$

L’iperparametro α bilancia il contributo delle due modalità guidate. Viene determinato tramite grid search su un set di validazione.

4 TIMO-S: Versione Migliorata di TIMO

TIMO-S è una variante potenziata di TIMO. Mentre TIMO ottimizza principalmente α , TIMO-S estende l’ottimizzazione includendo una grid search anche per gli iperparametri β (per TGI, che controlla il grado di guida testuale per l’immagine) e γ (per IGT, che controlla il grado di guida visiva per il testo) su un set di validazione. Questo permette di trovare una configurazione più robusta e ottimale per il grado di guida reciproca, portando a prestazioni ulteriormente migliorate. Il paper riporta che TIMO-S supera i metodi training-required esistenti con un costo computazionale significativamente inferiore.

5 Dataset Utilizzato

Per la valutazione di TIMO e TIMO-S, è stato impiegato un dataset derivato da una collezione più ampia di opere d’arte, comunemente nota come ArtGraph. La creazione del dataset specifico per gli esperimenti ha seguito una logica precisa per garantire la pertinenza e la gestibilità nell’ambito del few-shot learning.

5.1 Composizione del Dataset Originale

Il dataset di partenza è costituito da un vasto insieme di immagini di opere d’arte, ciascuna associata a metadati quali il nome dell’artista e lo stile artistico predominante. Queste informazioni sono tipicamente raccolte in un formato tabellare (un file CSV), dove ogni riga corrisponde a un’opera d’arte.

5.2 Creazione del Nuovo Dataset per la Sperimentazione

La costruzione del dataset utilizzato per gli esperimenti di classificazione few-shot, con l’obiettivo di classificare le opere per artista, ha comportato diversi passaggi di selezione e strutturazione:

1. **Selezione Basata sullo Stile e Numero di Opere:** Inizialmente, sono stati identificati un numero limitato dei stili artistici più rappresentati nel dataset originale (i primi 10 stili per numero di opere).

2. **Selezione degli Artisti per Stile:** All'interno di ciascuno di questi stili principali, sono stati selezionati un certo numero di artisti. Un criterio fondamentale per la selezione di un artista è stato il numero di opere possedute *all'interno di quello specifico stile*, che doveva rientrare in un intervallo predefinito (tra 30 e 50 opere). Questo assicura che ci siano abbastanza campioni per artista per creare split significativi, ma non così tanti da sbilanciare eccessivamente il dataset o allontanarsi troppo dallo scenario few-shot.
3. **Esclusione degli Altri Artisti:** È importante sottolineare che solo gli artisti che soddisfacevano i criteri di cui sopra (appartenenza a uno stile top e numero di opere adeguato in quello stile) sono stati inclusi nel dataset finale. Tutti gli altri artisti e le loro opere sono stati esclusi dalla sperimentazione.
4. **Divisione in Set di Support, Validazione e Query:** Per ciascun artista selezionato, le sue opere (appartenenti allo stile per cui è stato scelto) sono state suddivise percentualmente in tre insiemi distinti:
 - **Support set:** Una porzione maggioritaria delle opere (80%), utilizzata per "mostrare" al modello le caratteristiche dell'artista durante la fase di adattamento (support set nel few-shot learning).
 - **Validation set:** Una piccola porzione (10%), usata per l'ottimizzazione degli iperparametri del modello (come α , β , γ in TIMO/TIMO-S).
 - **Query set:** La restante porzione (10%), utilizzata per valutare le prestazioni finali del modello su dati mai visti (query set nel few-shot learning).
5. **Strutturazione Finale:** Il dataset risultante è stato organizzato in una struttura di directory standard, con le immagini raggruppate per artista e file di etichette separati che definiscono gli split di training, validazione e test. A ogni artista selezionato è stato assegnato un ID numerico univoco.

Questa metodologia di creazione del dataset mira a costruire un benchmark focalizzato e bilanciato per la classificazione few-shot di artisti, basandosi su una selezione ragionata degli stili e del numero di opere per artista.

5.3 Ruolo dei Prompt Testuali Generati da LLM

Nel contesto di modelli multimodali come CLIP, e quindi anche per TIMO, i **prompt testuali** giocano un ruolo cruciale. Termini come `Prompt_CuPL`, `Prompt_Waffle`, `Prompt_DCLIP`, ecc., si riferiscono a insiemi di descrizioni testuali generate per ogni classe del dataset (in questo caso, per ogni artista).

- **Generazione tramite Grandi Modelli Linguistici (LLM):** Questi prompt sono tipicamente prodotti utilizzando LLM. Per ogni artista, l'LLM viene istruito a generare molteplici frasi descrittive. Ad esempio, per un artista specifico, i prompt potrebbero includere varianti come "un dipinto di [Nome Artista]", "opere d'arte nello stile distintivo di [Nome Artista]", "arte creata da [Nome Artista]", ecc.
- **Scopo dei Prompt:**
 1. **Creare Rappresentazioni Testuali Ricche:** Fornire diverse descrizioni testuali per ogni artista aiuta a catturare un'ampia gamma di sfumature semantiche associate al suo lavoro e stile.
 2. **Migliorare la Generalizzazione di CLIP:** CLIP apprende ad associare immagini e testi. Utilizzando un insieme variegato di prompt per rappresentare testualmente un artista, si può ottenere una rappresentazione vettoriale (un "prototipo testuale") più robusta e generalizzabile per quell'artista.

3. **Supporto al Few-Shot/Zero-Shot Learning:** In scenari con pochi (o nessun) esempi di addestramento per artista, avere prototipi testuali di alta qualità è fondamentale. I prompt generati da LLM permettono di definire le classi al modello anche senza un gran numero di immagini di esempio.
4. **Guida per l'Ottimizzazione (come in TIMO):** Metodi come TIMO utilizzano questi prompt come punto di partenza. La componente IGT (Image-Guided Text) di TIMO, ad esempio, cerca di raffinare o pesare questi prompt testuali utilizzando le informazioni provenienti dalle immagini di supporto, al fine di migliorare l'allineamento tra le modalità visiva e testuale e, di conseguenza, le prestazioni di classificazione.

In sintesi, i prompt generati da LLM forniscono la necessaria controparte testuale che permette a modelli come CLIP di comprendere e classificare le immagini degli artisti, e costituiscono una componente essenziale per le strategie di adattamento e ottimizzazione impiegate in metodi avanzati come TIMO.

6 Risultati

6.1 Tabella

Tabella 1: Risultati Medi di Accuratezza (%) su ArtGraph. Ogni valore rappresenta la media delle accuratezze ottenute su 10 diversi seed per la specifica combinazione di Backbone, Numero di Shots e Modello di Adattamento.

Backbone	Shots	APE	GDA_CLIP	TIMO	TIMO_S	Tip_Adapter
RN101	1	42,39	41,68	45,90	45,90	38,47
	2	47,71	46,88	47,52	49,20	41,10
	4	52,39	55,66	57,89	58,26	45,44
	8	57,31	64,07	64,62	65,02	49,76
	16	59,05	71,28	71,13	71,31	54,25
RN50	1	44,28	41,41	43,88	46,09	38,65
	2	47,22	47,22	47,89	48,69	40,61
	4	53,85	55,54	56,48	57,28	44,95
	8	58,10	63,61	65,26	65,05	48,50
	16	62,17	73,06	73,36	73,49	52,54
ViT-B/16	1	50,89	48,93	52,39	53,58	47,06
	2	54,74	55,54	54,43	56,21	49,14
	4	60,28	63,15	64,25	64,74	52,57
	8	63,82	70,73	71,41	71,35	58,23
	16	68,47	77,71	78,10	78,01	62,66
ViT-B/32	1	45,41	43,36	47,52	48,13	41,47
	2	48,17	47,34	49,48	49,36	42,54
	4	55,02	56,67	59,33	59,45	45,90
	8	59,91	63,46	64,46	64,43	49,97
	16	62,91	70,43	71,28	71,28	56,97

6.2 Analisi dei Risultati

L'analisi dei risultati medi, ottenuti mediando le performance su 10 diversi seed per ciascuna configurazione, permette di delineare le seguenti osservazioni chiave:

1. Impatto del Numero di Shots (Campioni di Addestramento):

- Si osserva un incremento consistente dell'accuratezza media per tutti i backbone e modelli di adattamento all'aumentare del numero di "shots". Questo conferma l'efficacia di fornire più esempi specifici per il task target.
- I guadagni di performance sono generalmente più marcati nei passaggi iniziali (es. da 1 a 2 shots, o da 2 a 4 shots) rispetto agli incrementi con un numero maggiore di campioni (es. da 8 a 16 shots), suggerendo rendimenti decrescenti ma comunque positivi.

2. Performance dei Backbone Pre-addestrati:

- Il backbone ViT-B/16 emerge come il più performante in maniera predominante, raggiungendo le accuratèzze più elevate in quasi tutte le condizioni, specialmente con 16 shots dove supera il 78 % con i migliori metodi di adattamento.
- ViT-B/32 si posiziona come il secondo miglior backbone, superando i modelli ResNet ma rimanendo generalmente al di sotto di ViT-B/16, indicando un possibile vantaggio delle patch di dimensioni inferiori per questo task.
- I backbone RN50 e RN101 mostrano prestazioni competitive tra loro, ma sono tendenzialmente superati dai modelli ViT. La differenza tra RN50 e RN101 non appare così significativa come quella tra i ViT e i ResNet. In alcune configurazioni a 16 shots, RN50 eguaglia o supera leggermente RN101.

3. Efficacia dei Modelli di Adattamento Few-Shot:

- **Tip_Adapter** registra costantemente le prestazioni più basse, fungendo da baseline ma venendo nettamente superato dagli altri metodi.
- **APE** mostra un miglioramento rispetto a **Tip_Adapter**, ma si colloca in una posizione intermedia.
- I modelli **GDA_CLIP**, **TIMO** e **TIMO_S** si dimostrano i più efficaci, contendendosi le prime posizioni.
 - *Con pochi shots (1-shot):* **TIMO_S** e **TIMO** tendono a mostrare un leggero vantaggio o a essere i migliori (es. **TIMO_S** con ViT-B/16 raggiunge 53.58 %).
 - *Con un numero maggiore di shots (specialmente 8 e 16):* Le performance di **GDA_CLIP**, **TIMO** e **TIMO_S** diventano molto ravvicinate. **TIMO** con ViT-B/16 e 16 shots ottiene la performance più alta in assoluto (78.10 %), seguito da vicino da **TIMO_S** e **GDA_CLIP**.
- La differenza tra **TIMO** e **TIMO_S** è generalmente minima, con **TIMO_S** che a volte presenta un piccolo margine con un numero esiguo di dati.

4. Combinazioni Ottimali e Performance Massime:

- Le accuratèzze più elevate si ottengono combinando il backbone ViT-B/16 con 16 shots e i modelli di adattamento **TIMO**, **TIMO_S** o **GDA_CLIP**, superando la soglia del 78 %.
- Anche con 8 shots, la combinazione di ViT-B/16 con questi tre modelli di adattamento permette di raggiungere o superare il 70 % – 71 % di accuratezza, indicando una buona efficienza dei campioni.

5. Considerazioni Generali:

- La mediazione dei risultati su 10 diversi seed fornisce una stima più robusta e affidabile delle prestazioni, riducendo l'impatto della variabilità stocastica.
- Emerge chiaramente la sinergia tra un backbone potente (come ViT-B/16) e un metodo di adattamento few-shot efficace per massimizzare le performance.
- Nonostante i miglioramenti, il dataset **artgraph** si conferma sfidante, con le migliori accuratèzze che si attestano intorno al 78 % anche con 16 shots, suggerendo margini per ulteriori progressi.

A Pseudo-codice di TIMO

L'algoritmo 1 del paper originale fornisce uno pseudo-codice in stile PyTorch per l'algoritmo principale di TIMO.

Algorithm 1 Algoritmo principale di TIMO

```

1: Input: Feature immagini di supporto  $\mathbf{F}_v \in \mathbb{R}^{N \times K \times D}$ , feature testuali  $\mathbf{F}_t \in \mathbb{R}^{N \times P \times D}$ ,
   feature immagine di query  $\mathbf{f}_v \in \mathbb{R}^D$ , 'ImageClassifierBuilder', 'TextClassifierBuilder'.
2: Parametri: Peso del branch multimodale  $\alpha$ , controllore del grado di fusione  $\gamma_{\text{IGT}}$  (chiamato
    $\gamma$  nel paper per IGT, qui  $\gamma_{\text{IGT}}$  per chiarezza), controllore grado di guida testuale  $\beta_{\text{TGI}}$ 
   (chiamato  $\beta$  nel paper per TGI).

3: // Modellazione
4:  $\mathbf{F}_{vp} \leftarrow \mathbf{F}_v.\text{mean}(1).\text{unsqueeze}(1)$  ▷ Prototipi visivi per classe
5:  $\mathbf{F}_{vp} \leftarrow \mathbf{F}_{vp}/\mathbf{F}_{vp}.\text{norm}(-1, \text{keepdim}=\text{True})$  ▷ Normalizzazione L2
6:  $\mathbf{W}_{\text{sim}} \leftarrow \text{CosineSimilarity}(\mathbf{F}_t, \mathbf{F}_{vp}, -1)$  ▷ Pesi di similarità per TGI e IGT

7: // Costruzione Classificatore TGI
8:  $\mathbf{S}_{\text{TGI}} \leftarrow \text{SelezionaPesi}(\mathbf{W}_{\text{sim}}, \beta_{\text{TGI}})$  ▷ Selezione/pesatura basata su  $\beta_{\text{TGI}}$ 
9:  $\mathbf{F}_{\text{TGI}} \leftarrow \text{concat}([\mathbf{F}_v, \mathbf{F}_t \odot \mathbf{S}_{\text{TGI}}], 1)$  ▷ è prodotto element-wise o selezione
10:  $\text{Classifier}_{\text{TGI}} \leftarrow \text{ImageClassifierBuilder}(\mathbf{F}_{\text{TGI}})$ 

11: // Costruzione Classificatore IGT
12:  $\mathbf{R}_{\text{IGT}} \leftarrow \text{softmax}(\gamma_{\text{IGT}} \cdot \mathbf{W}_{\text{sim}}/\mathbf{W}_{\text{sim}}.\text{norm}(-1, \text{keepdim}=\text{True}), -1)$ 
13:  $\mathbf{F}_{\text{IGT}} \leftarrow \text{einsum}(\text{"np, npd -> nd"}, \mathbf{R}_{\text{IGT}}, \mathbf{F}_t)$  ▷ Feature testuali rettificate
14:  $\text{Classifier}_{\text{IGT}} \leftarrow \text{TextClassifierBuilder}(\mathbf{F}_{\text{IGT}})$ 

15: // Inferenza
16:  $\text{logits}_{\text{TGI\_out}} \leftarrow \text{Classifier}_{\text{TGI}}(\mathbf{f}_v)$ 
17:  $\text{logits}_{\text{IGT\_out}} \leftarrow \text{Classifier}_{\text{IGT}}(\mathbf{f}_v)$ 
18:  $\text{logits} \leftarrow \text{logits}_{\text{TGI\_out}} + \alpha \cdot \text{logits}_{\text{IGT\_out}}$ 
19: return logits

```

Nota: Lo pseudo-codice fornito nel paper è una versione semplificata. Ad esempio, $\mathbf{W}_{\text{sim}}$ qui rappresenta i pesi s^i per TGI e i pesi r^i (prima della softmax) per IGT. β_{TGI} e γ_{IGT} controllano il modo in cui questi pesi vengono usati.